Índice

1. Resumen ejecutivo y veredicto 1

2. Método de diagnóstico en vivo 1

3. El problema real: cadena completa del fallo 2

3.1 Por qué el despliegue se queda colgado (deploy infinito) . . . 2

4. Solución paso a paso (prioridad máxima) 3

Paso 1 — Aplicar la migración desde el SQL Editor de Supabase . . . 3

Paso 2 — Cancelar el deploy colgado y relanzar . . . 3

Paso 3 — Cambiar DATABASE_URL al pooler IPv4 (recomendado) . . . 4

Paso 4 — Variables de entorno recomendadas en Vercel . . . 4

Paso 5 — Verificación final . . . 4

5. Auditoría del sistema de correo (componente a componente) 5

6. Estado general de la plataforma 6

7. Hallazgos y recomendaciones 6

8. Limpieza del repositorio (v21) 7

9. Próximos pasos (lista de comprobación) 8

1. Resumen ejecutivo y veredicto

Se solicitó una auditoría completa de la web para determinar por qué los registros de nuevos usuarios «no llegan a la base de datos», con instrucción expresa de averiguar si la causa está en la base de datos o en la configuración de las variables de entorno, y de revisar en profundidad el sistema de envío de correos. Para responder con hechos y no con hipótesis, la auditoría combinó tres fuentes de evidencia: probes HTTP en vivo contra el entorno de producción (www.preureal.com), la lectura línea a línea del código del sistema de correo (rutas, librería de envío, migraciones y esquema) y el análisis del despliegue y su historial reciente.

La conclusión es nítida. Las variables de entorno están bien: la aplicación conecta con la base de datos en producción, el proveedor de correo (Resend) está configurado y el registro de usuarios funciona. La base de datos también está sana como servicio; lo que está desactualizado es su **esquema**: la migración 20261104120000_emaillog_telemetry, que añade las columnas que el sistema de correo v19 necesita (type, userId, attempts, sentAt, nextRetryAt, idempotencyKey), nunca llegó a aplicarse en Supabase. Cada envío intenta escribir su fila de «outbox» en la tabla EmailLog, encuentra columnas inexistentes y falla en silencio. Resultado: sin correos y sin filas nuevas en EmailLog, que es exactamente el síntoma reportado.

200 OK	500	200 OK
GET /api/health — la BD conecta (1015 ms)	GET /api/cron/emails — columnas nuevas ausentes	POST /api/auth/register — usuario creado en User

> **Veredicto:** ni las variables de entorno ni la conexión a la base de datos son la causa. El problema real es doble: (1) el **esquema de la base de datos** no tiene aplicada la migración del correo v19, y (2) el **despliegue v20**, que la aplicaría automáticamente durante el build, queda colgado porque el host directo de Supabase solo responde por IPv6 y el entorno de build de Vercel no alcanza esa dirección. La solución definitiva ocupa 2 minutos y está en el capítulo 4.

2. Método de diagnóstico en vivo

El diagnóstico se ejecutó contra producción, sin tocar datos de usuarios reales y sin necesidad de credenciales: basta con endpoints públicos y con consultas DNS. Esta vía es la más fiable porque observa el sistema en su estado real, no en una copia. Cada prueba se diseñó para separar una hipótesis concreta de las demás: si el servicio está caído, si la aplicación conecta con la base de datos, si el código desplegado es el esperado, y si el esquema de la base de datos está al día.

Prueba	Resultado	Interpretación
GET /api/health	HTTP 200 · database.ok · 1015 ms	La app y la conexión a PostgreSQL funcionan; no es un problema de variables de entorno ni de credenciales.
GET /api/cron/emails	HTTP 500 con cuerpo vacío	La ruta existe (código v19 desplegado), pero su primera consulta usa columnas (attempts, nextRetryAt, type) que no existen aún: el esquema está desactualizado.
POST /api/auth/register	HTTP 200 · usuario creado	El registro SÍ escribe la fila en la tabla User. La queja «no llega a la base de datos» se refería a EmailLog, alimentada por el correo.
DNS del host de la BD	db.…supabase.co responde SOLO IPv6	El runtime de Vercel llega por IPv6, pero el build de Vercel no: por eso el despliegue se queda «infinito» en prisma migrate deploy.
Build de Vercel (logs del usuario)	Colgado tras npm install	El deploy v20 nunca llegó a aplicar la migración; producción sigue con el esquema antiguo y el correo bloqueado.

Tabla 1. Evidencias recogidas contra producción (05/11/2026).

Con estas cinco pruebas quedan descartadas las tres hipótesis que más ruido generan en un caso así: caída del proveedor, error de credenciales y fallo de la aplicación. Queda una sola explicación compatible con todos los síntomas, y es la que desarrolla el capítulo siguiente.

3. El problema real: cadena completa del fallo

La versión v19 introdujo un sistema de correo robusto con «outbox» duradero: antes de enviar, cada correo crea una fila en EmailLog con estado «pending»; si el envío tiene éxito pasa a «sent» con el identificador de Resend, y si falla queda registrado para que el cron lo recupere. Para ello, el esquema necesita columnas nuevas que se añaden con una migración aditiva e idempotente. El código desplegado en producción es el correcto; el esquema de la base de datos, no: la migración sigue sin aplicarse.

La cadena del fallo es esta: el usuario se registra; la fila de usuario se crea correctamente en User (por eso el registro «funciona»); después, en segundo plano, el envío de bienvenida y de verificación intenta crear su fila de outbox en EmailLog; la consulta pide columnas que no existen (type, attempts, nextRetryAt…); PostgreSQL rechaza la operación; el error ocurre dentro del trabajo en segundo plano (after) y no interrumpe la respuesta, así que nadie lo ve; y el usuario no recibe ningún correo. Como nadie puede pedir el enlace de verificación, el síntoma se percibe como «los registros no llegan a la base de datos».

3.1 Por qué el despliegue se queda colgado (deploy infinito)

La v20 incluyó una salvaguardia importante: el build ejecuta «prisma migrate deploy» para que la base de datos nunca quede desactualizada. Esa salvaguardia es correcta, pero tropieza con un detalle de red: el host directo de la base de datos (db.tkbjifmflpxqszaozqlw.supabase.co) solo tiene dirección IPv6. El runtime de Vercel sí usa IPv6 y por eso la aplicación funciona; el entorno de build, en cambio, no alcanza ese host, y la

conexión de la migración queda esperando indefinidamente: el deploy entra en bucle y nunca aplica la migración. No es un fallo del código ni de Prisma: es un desajuste entre el endpoint de la base de datos y el entorno que debe alcanzarlo durante el build.

La consecuencia práctica es que el arreglo no consiste en reintentar el deploy una y otra vez: mientras el esquema no se ponga al día por otra vía (o mientras no se use el pooler IPv4), cada build volverá a colgarse en el mismo punto. El capítulo siguiente ordena las acciones en la secuencia exacta para resolverlo de forma definitiva.

4. Solución paso a paso (prioridad máxima)

Las dos primeras acciones desbloquean el correo en menos de cinco minutos y no requieren tocar código. Las restantes consolidan el arreglo para que el problema no pueda repetirse. Ejecútalas en orden.

Paso 1 — Aplicar la migración desde el SQL Editor de Supabase

Entra en Supabase, proyecto tkbjifmflpxqszaozqlw, abre **SQL Editor**, pega el siguiente SQL y pulsa **Run**. Es idempotente: puede ejecutarse varias veces sin error, y solo añade columnas e índices que faltan (no borra ni renombra nada).

```
ALTER TABLE "EmailLog" ADD COLUMN IF NOT EXISTS "type" TEXT NOT NULL DEFAULT 'GENERIC';
ALTER TABLE "EmailLog" ADD COLUMN IF NOT EXISTS "userId" TEXT;
ALTER TABLE "EmailLog" ADD COLUMN IF NOT EXISTS "providerMessageId" TEXT;
ALTER TABLE "EmailLog" ADD COLUMN IF NOT EXISTS "attempts" INTEGER NOT NULL DEFAULT 0;
ALTER TABLE "EmailLog" ADD COLUMN IF NOT EXISTS "sentAt" TIMESTAMP(3);
ALTER TABLE "EmailLog" ADD COLUMN IF NOT EXISTS "nextRetryAt" TIMESTAMP(3);
ALTER TABLE "EmailLog" ADD COLUMN IF NOT EXISTS "idempotencyKey" TEXT;
ALTER TABLE "EmailLog" ADD COLUMN IF NOT EXISTS "updatedAt" TIMESTAMP(3) NOT NULL
DEFAULT CURRENT_TIMESTAMP;

CREATE INDEX IF NOT EXISTS "EmailLog_status_nextRetryAt_idx" ON "EmailLog"("status",
"nextRetryAt");
CREATE INDEX IF NOT EXISTS "EmailLog_createdAt_idx" ON "EmailLog"("createdAt");
CREATE INDEX IF NOT EXISTS "EmailLog_userId_idx" ON "EmailLog"("userId");
CREATE UNIQUE INDEX IF NOT EXISTS "EmailLog_idempotencyKey_key" ON
"EmailLog"("idempotencyKey");
```

Código 1. Migración 20261104120000_emaillog_telemetry (v19/v20, idempotente).

Con el esquema al día, el código ya desplegado empieza a funcionar de inmediato: no hace falta redeploy para que los correos vuelvan a salir. Este es el paso que resuelve el incidente.

Paso 2 — Cancelar el deploy colgado y relanzar

En Vercel, entra en Deployments, cancela el deployment atascado (menú de los tres puntos, Cancel Deployment) y haz Redeploy del más reciente. Con la migración ya aplicada a mano, el paso «prisma migrate deploy» del build debería encontrar la base de datos al día. Si el build volviera a quedarse colgado en ese punto, aplica el Paso 3 antes de reintentarlo de nuevo.

Paso 3 — Cambiar DATABASE_URL al pooler IPv4 (recomendado)

Para que los builds de Vercel puedan conectar siempre con la base de datos, sustituye el valor de DATABASE_URL en Vercel por el **Session Pooler** de Supabase (host aws-0-región.pooler.supabase.com, puerto 5432, usuario postgres.tkbjifmflpxqszaozqlw), que sí tiene IPv4. Lo obtienes en Supabase con el botón Connect; sustituye el marcador [YOUR-PASSWORD] por la contraseña actual de la base de datos y añade &connect;_timeout=15 al final de la URL, para que un fallo de conexión sea visible en 15 segundos en lugar de colgar el build. No es necesario hacerlo para que el correo funcione (eso ya lo resuelve el Paso 1), pero evita que el deploy infinito vuelva a ocurrir en el futuro.

Paso 4 — Variables de entorno recomendadas en Vercel

Variable	Valor recomendado	Para qué
NEXT_PUBLIC_SITE_URL	https://www.preureal.com	Enlaces de los correos al dominio bueno (preureal.cat está sin DNS).
MAIL_FROM	PreuReal.cat	Con onboarding@resend.dev, Resend solo entrega a tu propio Gmail.
CRON_SECRET	Cadena larga aleatoria	Protege /api/cron/emails, que Vercel Cron llama una vez al día (04:00).
EMAILLOG_RETENTION_DAYS	90 (opcional)	Retención del log de correos que aplica el cron diario.

Tabla 2. Variables de entorno a definir o revisar en Vercel (Production).

Paso 5 — Verificación final

Comprueba, en este orden: que https://www.preureal.com/api/health sigue respondiendo 200; que /api/cron/emails ya devuelve JSON con «ok: true» (con 403 si defines CRON_SECRET, también es correcto); registra un usuario nuevo con un correo distinto de los anteriores; verifica en Supabase que aparecen su fila en User y, a los pocos segundos, las filas de bienvenida y verificación en EmailLog con status «sent» o «pending»; y revisa la bandeja de entrada y el spam. A partir de aquí, cada registro debe generar exactamente dos filas nuevas en EmailLog.

5. Auditoría del sistema de correo (componente a componente)

El sistema de correo v19/v21 está bien construido: es un outbox duradero con reintentos, idempotencia y seguridad de tokens. La auditoría línea a línea no ha encontrado defectos funcionales en el código desplegado; el único fallo operativo era el esquema desactualizado, ya resuelto con la migración. La tabla siguiente resume el estado de cada componente y su valoración.

Componente	Estado	Detalle de la revisión
Librería de envío (mail.ts)	Correcto	3 intentos en línea con backoff, guardián de 15 s por intento, errores desglosados con la causa real; el cuerpo guardado va siempre redactado.
Registro (register)	Correcto (mejorado en v21)	Crea User + token hasheado y envía bienvenida y verificación en segundo plano; en v21 ambos correos van en paralelo para no robar tiempo entre sí.
Verificación (verify)	Correcto	Token con hash en BD, confirmación atómica (solo gana una petición concurrente), un solo enlace vivo por cuenta y reenvío limitado por IP y usuario.
Restablecimiento (password-reset)	Correcto	Consumo atómico del token, anulación de otros enlaces pendientes y revocación de todas las sesiones al cambiar la contraseña.
Cron /api/cron/emails	Correcto (bloqueado por el esquema)	Reintenta el outbox, purga tokens caducados, aplica retención y redacta cuerpos legados; se protegerá con CRON_SECRET.
Idempotencia y telemetría	Correcto	idempotencyKey evita correos duplicados en reentregas de webhooks; providerMessageId y sentAt permiten auditar cada envío.

Tabla 3. Resultado de la revisión del sistema de correo.

Dos observaciones de mejora continua. Primera: el estado «sent» significa que Resend aceptó el correo; la entrega real en bandeja podría seguirse con los eventos de Resend (delivered, bounce, complaint) mediante un webhook, hoy inexistente. Segunda: VERIFY y RESET no se reenvían desde el log por diseño (el cuerpo guardado va redactado y el enlace es de un solo uso); si un correo de verificación se pierde, el usuario pide uno nuevo desde la interfaz, que es el comportamiento esperado.

6. Estado general de la plataforma

Más allá del correo, la auditoría recogió el estado de las integraciones que la plataforma declara en /api/health. El núcleo (mapa, datos, base de datos) está sano; hay dos integraciones sin configurar que conviene activar cuando toque, y una degradada que no afecta al uso principal.

Integración	Estado	Comentario
Base de datos PostgreSQL	OK	Conecta en ~1 s; latencia normal para Supabase.
ICGC (mapa y edificios)	OK	API Territorial respondiendo 200.
Correo (Resend)	OK	Configurado; pendiente verificar dominio propio.
Catastro (año de construcción)	Degradado	Consulta_DNPPC devuelve 404 desde producción; el sistema degrada con elegancia (yearBuilt vacío) y el mapa sigue funcionando. Revisar endpoint/cabeceras del servicio (P2).
Stripe (pagos)	Sin configurar	Sin claves en el entorno: el checkout no puede operar todavía.
Google (acceso con cuenta Google)	Sin configurar	Faltan GOOGLE_CLIENT_ID y GOOGLE_CLIENT_SECRET en Vercel.

Tabla 4. Estado de integraciones según /api/health y configuración declarada.

7. Hallazgos y recomendaciones

Se priorizan por impacto. Los P1 deben hacerse esta semana; los P2 son mejora continua. Ninguna recomendación requiere decisiones de arquitectura: son ajustes de configuración y operativa.

P1 — Seguridad y configuración (esta semana):

1. Rotar las credenciales expuestas en sesiones de trabajo anteriores: contraseña de la base de datos en Supabase (y actualizar DATABASE_URL en Vercel y en el .env local) y contraseña del panel de administración. Es la medida de mayor valor en seguridad y no interrumpe el servicio si se hace con orden.

2. Definir NEXT_PUBLIC_SITE_URL=https://www.preureal.com, MAIL_FROM con el dominio verificado en Resend y CRON_SECRET (Tabla 2).

3. Verificar el dominio preureal.com en Resend (registros DNS que indica el panel) para poder enviar a cualquier destinatario y no solo al correo propietario de la cuenta.

4. Usar siempre www.preureal.com en enlaces y materiales: el dominio preureal.cat está registrado pero sin registros DNS (NXDOMAIN), y un enlace muerto deja al usuario bloqueado en la verificación.

P2 — Mejora continua:

1. Revisar el endpoint del Catastro (404 actual) o darlo de baja del panel de estado si el servicio ha cambiado; el resto de la app no depende de él.

2. Añadir los eventos de Resend (delivered/bounce/complaint) como webhook para medir la entrega real en bandeja y alertar de rebotes.

3. Revisar manualmente las 25 vulnerabilidades que reporta npm audit (nunca usar npm audit fix --force) y actualizar por dependencias.

4. Monitorizar EmailLog y el resultado del cron diario como parte de la rutina de operación; son el panel de salud del correo.

8. Limpieza del repositorio (v21)

Se solicitó eliminar archivos no útiles o no funcionales. Se ha hecho con criterio conservador: nada que use el build de Vercel, ninguna función del aplicativo y ninguna referencia viva. Se eliminaron documentos de eras anteriores superados por VERCEL.md, CHANGES.md y README.md, ejemplos de tecnologías ajenas al proyecto y un Caddyfile relicario del autoalojamiento; y se des-trackeó del repositorio git la infraestructura local del entorno de desarrollo (184 MB de datos de una base de datos embebida y utilidades de sesión), que jamás debió versionarse. El paquete de despliegue queda en 160 ficheros y 0,36 MB, escaneado sin secretos.

Categoría	Elementos	Motivo
Documentación obsoleta	AUDIT.md, DEPLOY.md, INFORME_FINAL.md, TASKS.md, PROJECT_CONTEXT.md, MIGRATION_ZAI.md, COM_APPLICAR_ELS_CANVIS.md, AUDITORIA_FASE1_REGISTRE.md, CORRECCIONS_P1.md	Superados por VERCEL.md + CHANGES.md; el historial queda en git.
Infraestructura ajena	Caddyfile, examples/websocket, mini-services/	Restos del autoalojamiento y demos sin uso en el producto.
Infra local des-trackeada	.localpg (184 MB), skills/, download/, upload/, tool-results/, .zscripts/	Utilidades del entorno de desarrollo; añadidas a .gitignore y fuera del repositorio.

Tabla 5. Limpieza aplicada en v21 (ninguna función del producto fue tocada).

9. Próximos pasos (lista de comprobación)

La lista ordena todo lo pendiente del lado del propietario, con los paneles web como única herramienta. Los pasos 1 y 2 resuelven el incidente; el resto consolida.

1. Ejecutar el SQL del Código 1 en Supabase (SQL Editor → Run).
2. En Vercel: cancelar el deploy colgado y Redeploy; si se atasca de nuevo en prisma migrate deploy, aplicar el Paso 3 (Session Pooler + connect_timeout) y reintentar.
3. Definir en Vercel las variables de la Tabla 2 y hacer un Redeploy final.
4. Verificar: /api/health, registro de un usuario nuevo, filas en User y EmailLog, correo en bandeja (revisar spam).
5. Descargar el ZIP v21 (PreuReal-codi-complet-v21.zip) y desplegarlo con el flujo habitual: descomprimir, rsync al proyecto (excluyendo node_modules, .env, .env.local, .next y .git), npm install, build, commit y push.
6. Durante la semana: rotar credenciales (P1.1), verificar el dominio en Resend y activar CRON_SECRET si aún no lo está.

Si cualquier comprobación del paso 4 da un resultado distinto al esperado, el siguiente dato diagnóstico está siempre en la tabla EmailLog (columna error) y en los Runtime Logs de Vercel de la función correspondiente; con esa salida, cualquier incidencia posterior se puede aislar en minutos. Con la migración aplicada y el pooler configurado, la plataforma queda con su sistema de correo completo, seguro y observable.